



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 数据结构与算法导论

实验名称: 排序的编程与实现

一、实验目的

掌握常用的排序方法，并掌握 C 语言实现排序算法的方法；了解各种排序方法过程及依据原则，并掌握各种排序方法的时间复杂度和稳定性的分析方法。

二、实验原理

参照课本 p. 165-186, Figure 7. 2, 7. 4, 7. 8-7. 10, 7. 12-7. 16

排序是数据结构中的一种基本操作，其目的是按照某种关键字的大小关系，将一组数据重新排列成有序序列。本实验的任务是对学生成绩进行排序，每条学生信息由姓名和分数组成，要求按照分数从高到低输出学生名次，并且分数相同的学生名次相同。实验要求需要从键盘输入学生成绩数据，并采用一种常用排序算法完成成绩排名和结果输出。

本实验采用选择排序算法实现成绩排序。选择排序的基本思想是：每一趟从未排序的数据中选出分数最高的学生，将其放到当前未排序部分的最前面。第一趟选出最高分放到第一个位置，第二趟从剩余学生中选出最高分放到第二个位置，依次进行，直到所有学生按照成绩从高到低排列完成。

选择排序的主要步骤如下：首先设当前位置为最大值位置，然后依次与后面的学生成绩进行比较；如果发现后面某个学生的分数更高，则记录该学生的位置；一趟比较结束后，将当前学生与最高分学生进行交换。经过多趟排序后，学生成绩表就会按照分数降序排列。

排序完成后，还需要计算学生名次。第一个学生成绩最高，名次为 1；从第二个学生开始，如果当前学生的成绩与前一个学生相同，则名次相同；如果成绩不同，则名次为当前位置序号加 1。这样可以保证相同分数的学生获得相同名次，符合实验要求。

选择排序的时间复杂度为 $O(n^2)$ ，适合数据量较小的成绩排序问题。虽然它的效率不如快速排序等高级排序算法，但实现过程简单，逻辑清晰，便于理解排序算法的基本思想。

三、实验内容

统计成绩：给出 n 个学生的考试成绩表，每条信息由姓名和分数组成，利用排序算法完成以下任务，同时要求学生的考试成绩需要从键盘输入数据建立，同时要设计输出格式：

- 1) 按照分数高低次序，打印出每个学生在考试中获得的名次，分数相同的为同一名次
- 2) 按照名次列出每个学生的姓名和分数

四、实验要求

- 1) 能够采用常用的排序算法中的一种实现以上两个任务

五、实验源程序

```
#include <stdio.h>
#include <stdlib.h>
```

```

#define MAX_STU 100
#define NAME_LEN 30

typedef struct {
    char name[NAME_LEN];    // 学生姓名
    int score;               // 学生成绩
    int rank;               // 学生名次
} ScoreInfo;

/* 输入学生成绩信息 */
void InputScore(ScoreInfo stu[], int *num) {
    int i;

    printf("请输入学生人数: ");
    scanf("%d", num);

    while (*num <= 0 || *num > MAX_STU) {
        printf("人数输入不合法, 请重新输入 1-%d 之间的整数: ", MAX_STU);
        scanf("%d", num);
    }

    printf("\n 请依次输入学生姓名和成绩: \n");
    for (i = 0; i < *num; i++) {
        printf("第 %d 个学生: ", i + 1);
        scanf("%s %d", stu[i].name, &stu[i].score);

        while (stu[i].score < 0 || stu[i].score > 100) {
            printf("成绩应在 0-100 之间, 请重新输入该学生成绩: ");
            scanf("%d", &stu[i].score);
        }

        stu[i].rank = 0;
    }
}

/* 按成绩从高到低进行选择排序 */
void SelectSort(ScoreInfo stu[], int n) {
    int i, j, maxIndex;
    ScoreInfo temp;

    for (i = 0; i < n - 1; i++) {
        maxIndex = i;
    }
}
    
```

```

        for (j = i + 1; j < n; j++) {
            if (stu[j].score > stu[maxIndex].score) {
                maxIndex = j;
            }
        }

        if (maxIndex != i) {
            temp = stu[i];
            stu[i] = stu[maxIndex];
            stu[maxIndex] = temp;
        }
    }
}

/* 计算名次：分数相同者名次相同 */
void SetRank(ScoreInfo stu[], int n) {
    int i;

    if (n <= 0) {
        return;
    }

    stu[0].rank = 1;

    for (i = 1; i < n; i++) {
        if (stu[i].score == stu[i - 1].score) {
            stu[i].rank = stu[i - 1].rank;
        } else {
            stu[i].rank = i + 1;
        }
    }
}

/* 输出排名结果 */
void PrintScoreList(ScoreInfo stu[], int n) {
    int i;

    printf("\n 学生成绩排名表\n");
    printf("-----\n");
    printf("名次\t姓名\t\t成绩\n");
    printf("-----\n");

    for (i = 0; i < n; i++) {
        printf("%d\t%-10s\t%d\n", stu[i].rank, stu[i].name, stu[i].score);
    }
}

```

```

    }

    printf("-----\n");
}

int main() {
    ScoreInfo student[MAX_STU];
    int n;

    InputScore(student, &n);

    SelectSort(student, n);

    SetRank(student, n);

    PrintScoreList(student, n);

    return 0;
}

```

六、实验结果

请输入学生人数：5

请依次输入学生姓名和成绩：

第 1 个学生：Jack

87

第 2 个学生：Jim

96

第 3 个学生：Kael

99

第 4 个学生：Lily

77

第 5 个学生：Daney

95

学生成绩排名表

名次	姓名	成绩
1	Kael	99
2	Jim	96
3	Daney	95
4	Jack	87
5	Lily	77

七、实验心得

在程序编写过程中，我采用选择排序对学生成绩进行降序排列。选择排序每一趟都会从未排序部分中找出最高分学生，并将其放到前面。通过这一过程，我加深了对选择排序“每趟选择一个最值”的理解。与冒泡排序相比，选择排序不是频繁交换相邻元素，而是先找到最大值位置后再交换，因此交换次数相对较少，程序逻辑也比较清楚。

在名次计算部分，我认识到成绩排序完成后不能简单地按照 1、2、3、4 依次编号，因为实验要求分数相同的学生具有相同名次。因此在计算名次时，需要判断当前学生成绩是否与前一个学生相同：如果相同，则沿用前一个学生的名次；如果不同，则按照当前位置确定新的名次。这个处理过程让我进一步理解了算法设计中特殊情况判断的重要性。

通过本次实验，我不仅掌握了选择排序的基本实现方法，也熟悉了结构体数组在学生成绩管理中的使用。实验过程中，我对输入数据、排序过程、排名规则和格式化输出之间的关系有了更清晰的认识。总体来说，本次实验帮助我将排序算法与实际成绩统计问题结合起来，提高了使用 C 语言解决数据处理问题的能力。